

Build a Resume Matching Engine

Redrob AI Campus Hackathon — Individual Competition

Duration	Submission	Language	Allowed AI Tool	Competition Type	Libraries
60 Minutes	Google Form (LINK)	Any	Redrob AI only	Individual	Standard only

Problem Statement

You are given:

- 10 resume datasets from Indian university students
- 3 Job Descriptions (JDs) from Korean technology companies

Using **Redrob AI** as your coding assistant, build a program that:

- Normalizes noisy resume skill data
- Computes TF-IDF vectors for resumes
- Builds binary vectors for job descriptions
- Calculates cosine similarity between resumes and JDs
- Outputs the Top 3 matching candidates per JD

Implementation Workflow

Task	Description
Normalize Skills	Split skills on commas, lowercase tokens, apply alias mapping, and discard unknown tokens
Deduplicate Skills	Remove duplicate canonical skills from each resume
Build Vocabulary	Create a shared vocabulary from all normalized + deduplicated resume skills
Compute TF-IDF	Generate TF-IDF vectors for resumes using the exact formulas below
Build JD Vectors	Create binary vectors for JDs over the same vocabulary
Similarity & Ranking	Compute cosine similarity and rank the Top 3 candidates per JD

Step-by-Step Detail

Step 1: Skill Normalization

For every resume:

- Split raw skill strings using commas
- Convert every token to lowercase
- Match multi-word phrases before single-token matches
- Apply SKILL_ALIASES
- Discard tokens not present in the alias map

Normalization Example

Stage	Result
Raw Input	"Pyhton, Deep-learning, pandas, matplotlib"
Lowercase + Split	pyhton, deep-learning, pandas, matplotlib
Alias Mapping	python, deep_learning, pandas, data_visualization
Deduplicated Output	python, deep_learning, pandas, data_visualization

Step 2: Deduplication

After normalization:

- Each canonical skill may appear only once per resume

Step 3: Vocabulary Construction

Create a shared vocabulary from normalized, deduplicated resume skills only.

- Sort alphabetically
- Use same order for resume & JD vectors
- Ensures consistent cosine similarity results

Step 4: TF-IDF Vector Construction

Treat the 10 resumes as the complete corpus.

- Compute TF-IDF only for resumes
- Do NOT compute TF-IDF for JDs
- External libraries are prohibited

Reference Formulas

TF (Term Frequency)

$$TF(skill, resume) = \text{count}(skill \text{ in } resume) / \text{total_unique_skills}(resume)$$

After deduplication:
 $TF = 1 / N$

Where:

- N = total unique skills in the resume

IDF (Inverse Document Frequency)

$$IDF(skill) = \ln(10 / df(skill))$$

Where:

- $df(skill)$ = number of resumes containing the skill
- Total resumes = 10
- Natural logarithm must be used
- No smoothing allowed

TF-IDF

$$TF-IDF = TF \times IDF$$

Cosine Similarity

$$\text{Cosine}(A, B) = (A \cdot B) / (|A| \times |B|)$$

Where:

- A = Resume TF-IDF vector
- B = JD binary vector
- $|A|$ = Euclidean norm of A

Resume Dataset — 10 Candidates

ID	Candidate	Raw Skills	Background
01	Arjun Sharma	Pyhton, MachineLearning, SQL, pandas, numpy, Deep-learning	TCS Intern · BITS Pilani CSE 2024
02	Priya Nair	JavaScrpit, Reacts, Node.JS, MongoDB, REST api, HTML/CSS	Freelance Web Developer · VIT IT 2024
03	Rahul Gupta	Java, Spring Boot, MySql, Microservices, Docker, kubernates	Infosys SDE Intern · IIT Delhi 2023
04	Sneha Patel	Python, TensorFlow, Keras, NLP, BERT, data-viz, matplotlib	IISc Research Assistant · IIIT Hyderabad AI 2024
05	Vikram Singh	C++, Algoritms, Data Structure, competitive programming, python	Google SWE Intern · IIT Bombay 2024
06	Ananya Krishnan	javascript, vue.js, python, flask, PostgreSQL, AWS, CI/CD	Full Stack Developer · NIT Trichy 2022
07	Karan Mehta	Python, Sklearn, XGboost, feature engineering, SQL, tableau	Data Analyst · Delhi University 2023
08	Deepika Rao	Java, Android, Kotlin, Firebase, REST, UI/UX, figma	Samsung Android Intern · NSIT 2024
09	Aditya Kumar	Reactjs, TypeScrpit, GraphQL, redux, tailwind, nodejs, jest	Frontend SDE · Flipkart / IIIT Bangalore
10	Meera Iyer	python, R, statistics, ML, regression, clustering, Power-BI	Data Science Intern · Wipro 2024

Job Description Dataset

JD	Company	Role	Required Skills	Preferred Skills
JD-1	Kakao (Seoul)	ML Engineer	Python, Machine Learning, Deep Learning, TensorFlow, PyTorch, SQL, Data Visualization	NLP, BERT, Feature Engineering, Statistics
JD-2	Naver (Seongnam)	Backend Engineer	Java, Spring Boot, MySQL, PostgreSQL, Microservices, Docker, Kubernetes	REST API, CI/CD, Redis
JD-3	Line (Seoul)	Frontend Engineer	JavaScript, React, Vue, TypeScript, REST API, HTML/CSS	Node.js, GraphQL, Redux, Jest, AWS

SKILL_ALIASES (Use Exactly As Provided)

- Do not modify this mapping
- Do not add custom aliases
- Multi-word phrases must be matched before token-level processing
- Tokens not present in this mapping must be discarded

```
SKILL_ALIASES = {  
    # Languages  
    "python": "python",  
    "pyhton": "python",  
    "java": "java",  
  
    "javascript": "javascript",  
    "javascripit": "javascript",  
    "js": "javascript",  
  
    "typescript": "typescript",  
    "typescripit": "typescript",  
  
    "c++": "cpp",  
    "cpp": "cpp",  
    "r": "r",  
    "kotlin": "kotlin",  
  
    # ML / Data  
    "machinelearning": "machine_learning",  
    "machine learning": "machine_learning",  
    "ml": "machine_learning",  
    "sklearn": "machine_learning",  
  
    "deeplearning": "deep_learning",  
    "deep learning": "deep_learning",  
    "deep-learning": "deep_learning",  
  
    "tensorflow": "tensorflow",  
    "pytorch": "pytorch",  
    "keras": "keras",  
  
    "nlp": "nlp",  
    "bert": "bert",  
    "xgboost": "xgboost",  
  
    "feature engineering": "feature_engineering",  
  
    "statistics": "statistics",  
    "stats": "statistics",  
  
    "regression": "regression",  
    "clustering": "clustering",  
  
    "data-viz": "data_visualization",  
    "data visualization": "data_visualization",  
    "data viz": "data_visualization",  
    "matplotlib": "data_visualization",  
    "tableau": "data_visualization",  
    "power-bi": "data_visualization",  
}
```

```

"power bi": "data_visualization",
"powerbi": "data_visualization",

"pandas": "pandas",
"numpy": "numpy",

# Web - Frontend
"react": "react",
"reacts": "react",
"reactjs": "react",

"vue": "vue",
"vue.js": "vue",
"vuejs": "vue",

"redux": "redux",
"tailwind": "tailwind",

"html/css": "html_css",
"html css": "html_css",
"html": "html_css",
"css": "html_css",

"jest": "jest",
"graphql": "graphql",

# Web - Backend
"node.js": "nodejs",
"nodejs": "nodejs",
"node js": "nodejs",

"flask": "flask",

"spring boot": "spring_boot",
"springboot": "spring_boot",

"rest api": "rest_api",
"rest": "rest_api",
"restapi": "rest_api",

"microservices": "microservices",

# Databases
"sql": "sql",
"mysql": "mysql",
"mysq": "mysql",

"postgresql": "postgresql",
"postgres": "postgresql",

"mongodb": "mongodb",
"redis": "redis",

# DevOps / Cloud
"docker": "docker",

"kubernetes": "kubernetes",
"kubernates": "kubernetes",
"k8s": "kubernetes",

"ci/cd": "ci_cd",
"cicd": "ci_cd",
"ci cd": "ci_cd",

"aws": "aws",

```

```
# Mobile
"android": "android",
"firebase": "firebase",

# CS Fundamentals
"algorithms": "algorithms",
"algoritms": "algorithms",

"data structure": "data_structures",
"data structures": "data_structures",

"competitive programming": "competitive_programming",

# Design
"ui/ux": "ui_ux",
"ui ux": "ui_ux",
"figma": "figma",
}
```

Expected Output Format

```
JD-1 - Kakao (ML Engineer)
Name(score), Name(score), Name(score)

JD-2 - Naver (Backend Engineer)
Name(score), Name(score), Name(score)

JD-3 - Line (Frontend Engineer)
Name(score), Name(score), Name(score)
```

Output Rules

- Scores must be rounded to 2 decimal places
- Ties must be broken alphabetically by candidate name

Submission Requirements

Field	Description	Points
Personal Information	Name / University / Student ID	—
JD-1 Result	Top 3 ranked candidates with scores	6
JD-2 Result	Top 3 ranked candidates with scores	7
JD-3 Result	Top 3 ranked candidates with scores	7
Redrob AI Usage	Summary of prompts used — step-by-step scores higher	40
Full Code	Used for tiebreaking and top-20 review	20
F2F Discussion	Problem statement discussion	20

Scoring Criteria

Criteria	Points	How It's Evaluated
Output Accuracy	20	9 results checked against the answer key
Redrob AI Usage Quality	40	20 pts — 3+ staged prompts · 12 pts — 2 steps · 5 pts — single request
Code Quality	20	Top 20 participants only — structure, readability, correctness
F2F Discussion	20	Problem statement discussion

Rules & Guidelines

Allowed

- Redrob AI
- Standard libraries (math, collections, re, etc.)
- Internet browsing for documentation / reference

Not Allowed

- External libraries (numpy, pandas, scikit-learn, etc.)
- Other AI tools
- Sharing solutions with participants
- Multiple submissions
- Modifying SKILL_ALIASES

Tips for Participants

Strong submissions usually:

- Break the problem into stages
- Validate intermediate outputs
- Verify normalization carefully
- Inspect TF-IDF values before ranking

Recommended Redrob AI workflow:

- Explore noisy tokens
- Build normalization logic
- Deduplicate skills
- Compute TF-IDF manually
- Build JD vectors
- Compute cosine similarity and rankings

Note: Copying AI output without verification is the most common source of errors. Validate your normalization, deduplication, and intermediate TF-IDF values at each step before finalising your answer.

Google Form Submission

<https://forms.gle/Z9Qu7iuFzcc73Xr58>